GraphLang

A graph-first language for agentic workflows.

The graph is the program. The model edits the topology, not the code.

wen.gong.research@gmail.com

https://github.com/digital-duck/graflang

The problem with agent code

- An agent workflow is usually **Python glue** around prompts: loops, retries, parsing, state.
- You cannot **see** its shape without reading it all.
- You cannot **check** it before it spends money.
- When a model rewrites it, the model writes **arbitrary code**.

WHAT WE WANT

✔ the structure is data you can draw

✔ checks before anything runs

✔ hard limits on steps, calls, cost

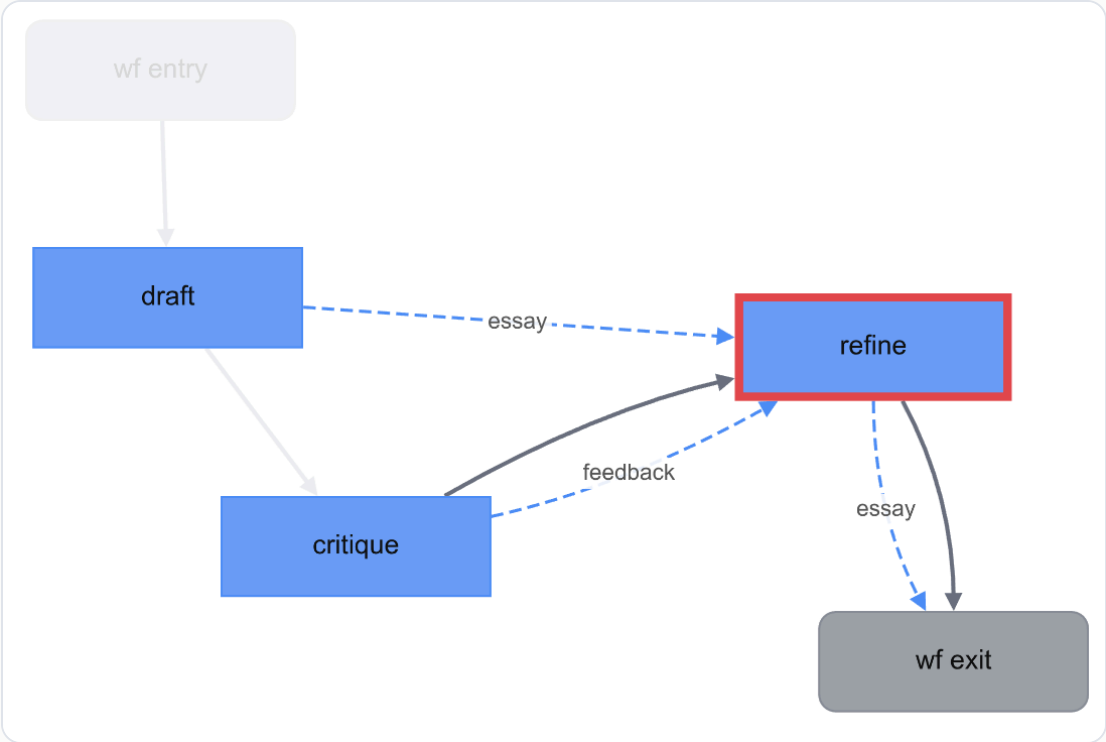
✔ a model may change the shape, but only inside rules

✔ **secure by design**: Cedar policies decide what a workflow may contain and what a model may change, and the check fails closed

The idea

A workflow is a YAML file: `NAME.graph.yaml`.

- **Nodes** do work: ask a model, test a condition, run a solver, loop.
- **Edges** say who fires next and where data flows.
- The runtime passes tokens along the edges. A node fires when a token reaches it.
- Evolved from SPL (Structured Prompt Language); a graph can be lifted from SPL and written back.



solid = control (who fires next) · dashed = data (what flows)
The picture `graf vis` draws for `cookbook/self_refine` (refine selected: its links stand out).

Two kinds of edge, ports for everything

```
# control: who fires next
- {source: "draft.complete",
   target: "critique.trigger",
   type: triggers}

# data: where a value flows
- {source: "draft.out_essay",
   target: "critique.in_essay",
   type: binds}
```

- `trigger` `complete` `pass` `fail` `entry` `exit` carry **control**.
- `in_x` and `out_x` carry **data**.
- In a prompt, `{{in_topic}}` is filled from that port.
- Conditions see only the node's own inputs: `len(@in_poem) > 10`.
- `graf validate` rejects a prompt that names a port nothing feeds.

The node types

TYPE	WHAT IT DOES
INPUT OUTPUT	the workflow's entry and exit; typed inputs and outputs
GENERATE	one model call from a prompt file
EVALUATE ASSERT	a condition routes the token to `pass` or `fail`
SOLVE	exact computation: an expression, a Python kernel, or a program run as a subprocess (optionally in a container)
LOOP FORK JOIN BATCH	repeat a sub-graph; fan out; wait for all; run once per item
GUARD MUTATE	a Cedar policy check; a model-proposed change to the graph, validated before it is applied
RUN CONTEXT	work done by an external program; shared state

Demo 1: run, look, read

> **LIVE** about 3 minutes; a handful of short model calls

```
graf info cookbook/self_refine/self_refine.graph.yaml
graf vis  cookbook/self_refine/self_refine.graph.yaml
graf run  cookbook/self_refine/self_refine.graph.yaml --param topic="health"
```

- `info` : name, size, inputs, outputs, resources.
- `vis` : click a node for its config and links; "show data edges" toggles the dashed edges.
- `run` : each node prints as it fires, then the output and the log path.

Default model is `claude_cli` with `claude-sonnet-5-5` ; no API key, the `claude` tool must work.

Write your own: a retry is a wire

```
nodes:
  - {id: start, type: INPUT,
     ports: [entry, out_topic]}
  - {id: write, type: GENERATE,
     config: {prompt_ref: prompts.haiku},
     ports: [trigger, complete, in_topic, out_poem]}
  - {id: gate,  type: EVALUATE,
     config: {condition: "len(@in_poem) > 10"},
     ports: [trigger, pass, fail, in_poem]}
  - {id: done,  type: OUTPUT,
     ports: [trigger, exit, in_result]}
edges:
  - {source: start.entry,    target: write.trigger, type: triggers}
  - {source: write.complete, target: gate.trigger,  type: triggers}
  - {source: gate.pass,      target: done.trigger,  type: triggers}
  # a failed check goes back: the retry
  - {source: gate.fail,      target: write.trigger, type: triggers}
```

`prompts/haiku.md`:

```
Write a haiku about: {{in_topic}}
```

`gate` sends the token to `pass` or `fail`. `fail` goes back to `write`: a bad result tries again.

Abridged: the binds (data) edges are in the tutorial.

Break it on purpose

> **LIVE** make the condition impossible, then add a typo

AN ENDLESS RETRY

```
condition: "len(@in_poem) > 100000"

graf run haiku.graph.yaml --max-steps 20
```

✗ Run failed: stopped after 20 node firings

names the nodes that fired most. `--max-steps` is the safety net.

A TYPO

```
condition: "len(@poem) > 10"

graf validate haiku.graph.yaml
```

✗ uses a variable that is not available

caught before anything runs, before any money is spent.

Let a LOOP own the repetition

```
- id: retry
  type: LOOP
  config:
    condition: "NOT @ok AND @cycle < 3"
    outputs:
      poem: {from:  write.out_poem}
      ok:   {fired: gate.pass}
    subgraph:
      nodes: [ write, gate ]   # a plain sequence
```

- Condition is checked **before** each iteration.
- `@cycle` counts from 0, with a built-in cap.
- `@ok` is last iteration's output.
- No back-edge inside the body: the loop supplies the repetition.
- A MUTATE inside a loop body changes only that iteration's copy.

After a run: logs, outputs, history

ON DISK

- **Log** per run in `~/.graflang/logs/` : a `.log` for people, a `.jsonl` for tools. Failed runs too.
- **Output folder** `output/<graph>-<model>-<time>/` : `result.json` , a file per text output, `summary.json` , and with `--save-nodes` every node's output.

```
jq -r 'select(.event=="llm.call")
  | [.node,.model,.ms] | @tsv' ~/.graflang/logs/*.jsonl
```

IN THE APP

- The Edit page's **Logs** tab shows a run readably: text as formatted text, structure as YAML.
- A picker lists **past runs** saved beside the graph, newest first, with model and call count.
- Per-node results, time per node, model usage.

Another model: change the command, not the graph

```
graf run haiku.graph.yaml --llm claude_cli:claude-haiku-5-5 --param topic="health"
```

- Adapter and model are chosen at run time. The file names of logs and outputs carry them, so runs with different models are easy to tell apart.
- Real calls go through **dd-llm**; a `--bind` file holds `limits`: `max_llm_calls`, `max_cost_usd`, `wall_time_s`.
- **Record and replay**: save the model's answers once, re-run for free.
- **Checkpoint and resume**: a run that fails at step N resumes from there.

A real example: PII compliance audit

1. LLM reads a document, lists the **types** of personal data (never the values).
2. rules decide, with no LLM, whether it may go to a destination.
3. A failing document is **redacted, re-read, re-checked** once.
4. Cedar GUARD has the last word.
5. Anything not shown compliant ends at `escalated`.

THE POINT

The model does the fuzzy reading. Exact rules decide. The graph **fails closed**.

With Haiku the same input ended at `escalated`: a false alarm, not a leak. That is the intended behaviour.

`graf validate` proves the risky output is reachable only through the guard's `allow` port.

> LIVE `graf run cookbook/compliance_audit/compliance_audit.graph.yaml --param document="…SSN…" --param destination=external_api --param engine=stdlib --param rules_file="./procedures/rules_extended.json"`

Exact work goes to solvers

SOLVE BACKENDS

- `expression`: a safe evaluator, no `eval`.
- `kernel`: a Python (or SageMath) kernel; variables persist across nodes.
- `subprocess`: a black-box program read through a declared parser.
- optional `container`: `docker run`, no network, read-only.

47 SOLVER RECIPES

Migrated from SPL: Z3, MILP, Nash and Stackelberg games, Lean, Sage, MiniZinc, Pyomo, SimPy, pandera and more.

All run with the default model; the fixes needed are listed in `cookbook_solvers/README.md`. Not installed here: Mathlib, Couenne/Bonmin, the MiniZinc binary (fallbacks used).

Safety by design: validate and policy

MECHANISM	WHAT IT GIVES YOU
graf validate	structure, ports, conditions, prompts with no feeding edge; a risky output reachable only through a guard
GUARD	**Cedar** policy via the official engine; fails closed (any evaluation error denies)
MUTATE	a model's graph change: schema, structure, **Cedar** policy, budget, then invariants on a trial copy. It can never remove a guard, and can never add code (`code` , `command` , `backend` …)
Limits	`--max-steps` , `max_llm_calls` , cost, wall time
Policy	`workflow.policy` : what a workflow may contain; programs on your computer need an explicit acknowledgement in the app

Executors are not a sandbox: see "What the executors do not confine" in the user guide.

Describe it, get a graph; read a graph, get words

CREATE (VIBE CODING)

```
graf create --name meeting_notes \
  --spec "turn raw notes into a summary,
  decisions and action items with
  owners; fix items with no owner"
```

A model writes `meeting_notes.graph.yaml`, prompts and a readme. It is checked like `validate`, plus a dry run, and retried up to 3 rounds.

DESCRIBE (REVERSE-ENGINEERING)

```
graf describe apps/meeting_notes
```

The graph in words: inputs, output, steps in order, data flow, cost. No model needed; with `--llm` one billed call writes a full specification.

That text is what `graf create --spec @FILE` takes: graph → spec → graph.

The app: `graf vis --edit`

> **LIVE** create an app from a sentence, then edit and run it

- **Welcome to GraphLang**: what this is and where to start.
- **Create**: chat with a model; the draft shows readme, workflow picture, graph and prompts.
- **Edit**: picture plus tabs: Source, Node, Add node, Edges, Problems, **Run**, **Logs**.
- **Browse**: search recipes by meaning and keywords.
- **Import / Export** zip, and **Settings**.

GOOD TO KNOW

Binds `127.0.0.1` by default: the app has no login.

API keys live in a 0600 settings file; an environment variable wins.

Zip import never overwrites and runs nothing.

Keep graphs logical and readable

COMPACT `STEPS:`

```
steps:
  - draft:    {generate: prompts.draft,
               in: {topic: $in.topic}, out: essay}
  - critique: {generate: prompts.critique,
               in: {essay: draft}, out: feedback}
  - refine:   {generate: prompts.refine,
               in: {essay: draft, feedback: critique}}
  - done:     {output: refine}
```

Control follows the order of the steps, so a straight line needs no edges. `graf expand FILE` prints the long form.

PARTS AND `EXPAND:`

- A **part** is a workflow used as a node, flattened in place before it runs.
- `expand:` applies a part to **every item** of a graph of things; edges become FORK / JOIN wiring.

Compile it to target language

```
graf compile workflow.graph.yaml --lang spl -o workflow.spl
graf compile workflow.graph.yaml --lang rust          # experimental stub
```

- **SPL**: the graph is written as an SPL script; round trip with `spl2graph` (which lives in SPL.py, so GraphLang does not depend on SPL).
- **Rust**: a stub only. Not a claim.
- Both run the `validate` checks first.

Experiments

The same code-pipeline workflow on 60 HumanEval tasks, three arms (single prompt, GraphLang, LangGraph), scored by one hidden-test oracle.

MODEL	SINGLE PROMPT	GRAPHLANG	LANGGRAPH
Sonnet 5.5	100%	98.3%	98.3%
Haiku	96.7%	93.3%	96.7%

What this shows: the graph form does not break the pipeline. GraphLang and LangGraph are equivalent here.

What it does not show: any accuracy gain, or that the pipeline is worth its cost: the single call is at least as accurate here and uses about 1/7 of the calls. No claim of fewer loops or faster audits is made, because no such experiment exists.

GraphLang vs LangGraph

GRAPHLANG LEANS ON

- **The program is data.** Validate before a run, draw it, diff it, review it without reading control flow.
- **Models edit a graph** through validated deltas, not by writing code; a policy says what they may change.
- Budgets and guards are in the file, not in a wrapper.
- Same pass rate and cost as LangGraph in our test.

LANGGRAPH LEANS ON

- **Mature and widely used:** a large ecosystem, integrations, docs, and people who already know it.
- **Plain Python:** any logic, any library, normal debugger and tests. Shorter here: 107 lines against 190.
- Fits an existing Python codebase.

Idea and plan

- **Safer self-editing agents:** a model reshapes its own workflow while the guard, budget and policy stay out of its reach (the experiment is not done).
- **Stronger isolation** for the programs a graph runs.
- **Formal checks** beyond races and deadlocks, on the same graph.
- **More targets:** a real Rust runtime, and export to other frameworks, if the graph stays the single source.
- **Lifting existing workflows** into graphs, so older code and scripts get the same checks.
- **A shared recipe library** you search by meaning, copy and adapt.

New node types:

OBSERVE: observability; record what happens without slowing the run.

QUERY: read from the world outside the graph.

EMIT: act on it, e.g. write a file or send a notification (email, text).

Try it - Apache 2 licensed

```
git clone https://github.com/digital-duck/graflang.git
cd graflang
pip install -e ".[dev]"
graf run  cookbook/self_refine/self_refine.graph.yaml \
        --param topic="health"
graf vis  --edit        # the app
graf create --name my_app --spec "…"
```

READ NEXT

- `docs/GUIDE/SETUP.md`
- `docs/GUIDE/TUTORIAL.md` : 15 minutes, hands on
- `docs/GUIDE/USER-GUIDE.md` : every command and node
- `cookbook/` and `cookbook_solvers/`

Questions?

The graph is the program.

Draw it, check it, bound it, then run it.

References

OUR WORK

- **GraphLang**: github.com/digital-duck/graflang
- **SPL** (the language GraphLang grew from): github.com/digital-duck/SPL.py
- W. G. Gong, *SPL: Orchestrating Workflows with Declarative Deterministic-Probabilistic Composition*, arXiv:2607.07727, 2026: arxiv.org/pdf/2607.07727

POLICY

- **Cedar** policy language and engine: github.com/cedar-policy
- Cutler et al., *Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization*, arXiv:2403.04651: arxiv.org/abs/2403.04651

IDEAS

- **Interaction nets** (the inspiration for token passing): wikiwand.com/en/Interaction_nets

COMPARED WITH, AND BENCHMARK

- **LangGraph**: github.com/langchain-ai/langgraph
- **PocketFlow**: github.com/The-Pocket/PocketFlow
- **HumanEval**: Chen et al., *Evaluating Large Language Models Trained on Code*, arXiv:2107.03374: arxiv.org/abs/2107.03374